

## Experiment 1 - Write a Python program to solve the 8-Puzzle Problem.

### Program Code

```
import heapq

def manhattan_distance(state, goal):
    distance = 0

    for i in range(9):
        if state[i] != 0:
            current_row = i // 3
            current_col = i % 3

            goal_index = goal.index(state[i])
            goal_row = goal_index // 3
            goal_col = goal_index % 3

            distance += abs(current_row - goal_row) + abs(current_col - goal_col)

    return distance

def get_neighbors(state):
    neighbors = []
    zero = state.index(0)

    row = zero // 3
    col = zero % 3

    moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
```

```
]
```

```
for dr, dc in moves:
```

```
    new_row = row + dr
```

```
    new_col = col + dc
```

```
    if 0 <= new_row < 3 and 0 <= new_col < 3:
```

```
        new_zero = new_row * 3 + new_col
```

```
        new_state = list(state)
```

```
        new_state[zero], new_state[new_zero] = new_state[new_zero],  
new_state[zero]
```

```
        neighbors.append(tuple(new_state))
```

```
return neighbors
```

```
def a_star(start, goal):
```

```
    priority_queue = []
```

```
    h = manhattan_distance(start, goal)
```

```
    heapq.heappush(priority_queue, (h, 0, start, []))
```

```
    visited = set()
```

```
    while priority_queue:
```

```
        f, g, state, path = heapq.heappop(priority_queue)
```

```
        if state in visited:
```

```
            continue
```

```
        visited.add(state)
```

```
        path = path + [state]
```

```
        if state == goal:
```

```
    return path
```

```
    for neighbor in get_neighbors(state):  
        if neighbor not in visited:  
            new_g = g + 1  
            new_h = manhattan_distance(neighbor, goal)  
            new_f = new_g + new_h  
  
            heapq.heappush(  
                priority_queue,  
                (new_f, new_g, neighbor, path)  
            )
```

```
    return None
```

```
def print_state(state):  
    for i in range(0, 9, 3):  
        print(state[i], state[i + 1], state[i + 2])  
    print()
```

```
start = (1, 2, 3,  
         4, 0, 6,  
         7, 5, 8)
```

```
goal = (1, 2, 3,  
        4, 5, 6,  
        7, 8, 0)
```

```
solution = a_star(start, goal)
```

```
if solution:  
    print("Solution found!")  
    print("Number of moves:", len(solution) - 1)  
    print()
```

```
    for step, state in enumerate(solution):
```

```
        print("Step", step)
        print_state(state)
else:
    print("No solution found.")
```

## Program Output

```
PS D:\AI LAB> py experiment1_8_puzzle.py
Solution found!
Number of moves: 2

Step 0
1 2 3
4 0 6
7 5 8

Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0
```